GitHub Copilot 워크플로 과정

PR · Code Review · CI/CD 연동

요구사항 하나를 문서에서 배포까지

1 요구사항

2 CI/CD 준비

3 PR · CI

4 Copilot 리뷰

5 머지 → 배포

요구사항에서 배포까지 — 누가 무엇을 맡나



01

요구사항 → Jira → 구현

요구사항은 Confluence 에 먼저 쓰고, Skill 로 Jira 작업을 만든 뒤 그 작업으로 구현까지 합니다.

실습

요구사항 → Jira → 구현 — Skill 두 번

`/req-to-jira` 로 요구사항 · Jira 를 만들고,
`/jira-implement` 로 feat/<Jira 키> 브랜치에서 구현 · 커밋

준비

- 새 세션마다 하네스 **Local** 선택 (Atlassian MCP)
- M5 끝에 push한 `feat/issue-2` 브랜치
- Atlassian API 토큰 (Jira 자동 갱신용)

실습 시작

출발 상태 · 자동화 파일 가져오기

1. `feat/issue-2` 인지 확인 (따라잡기 `feat/m6`)
2. `m6-start` 의 `.github` 가져와 커밋
3. Skill · 에이전트 `flow-tracker` · Jira 워크플로 · hook
4. VS Code **Developer: Reload Window**

명령 (터미널, 한 줄씩)

```
git status
git branch --show-current
# feat/issue-2 가 아니면: git switch feat/issue-2
# 따라잡기: git switch -c feat/m6 m6-start

git fetch origin
git checkout origin/m6-start -- .github
git commit -m "chore: add M6 automation"
git push
```

✅ **`.github/skills/jira-implement`** ✅ **`.github/workflows/jira-sync.yml`**

실습 시작-2

Jira 자동 갱신 시크릿

1. Atlassian 계정 설정 → **API tokens** → **Create API token**
2. 오른쪽 명령으로 저장소 시크릿 3개 · 변수 1개
3. 토큰은 마지막 명령의 입력창에만 — 파일 · 채팅 · 기록 금지
4. **Settings** → **Secrets and variables** → **Actions** 에서 확인

명령 (터미널, 한 줄씩)

```
gh secret set JIRA_BASE_URL -b "https://<사이트>.atlassian.net"
gh secret set JIRA_EMAIL -b "<이메일>"
gh variable set JIRA_PROJECT -b "KAN"
gh secret set JIRA_API_TOKEN
```

✅ 시크릿 3개 ✅ 변수 JIRA_PROJECT

실습 1-1

`/req-to-jira` — 요구사항 페이지 · Jira 작업

1. 새 세션(하네스 **Local**)에 오른쪽 한 줄
2. 쓰기 카드마다 Input 확인 → ▾ **Allow Once**
3. 끝에 다음 단계 `/jira-implement <키>`
4. 표: 페이지 URL · Jira 키 · 인수조건 개수

프롬프트 / 명령

```
/req-to-jira 알람 웹훅 알림 — CRITICAL
알람이 생기면 EQUIP_WEBHOOK_URL 로
알람 JSON 을 POST 한다. 실패하면 3번까지
다시 보내고, 그래도 실패해도 알람 저장 ·
조회는 그대로 동작한다. URL 이 없으면
보내지 않는다.
```

✅ **Confluence [요구사항] 페이지** ✅ **Jira 작업 · 양방향 링크** ✅ **Jira 설명에 Copilot 작업 지시**

실습 1-2

/jira-implement — 브랜치 · 진행 중 · 구현 · 커밋

1. 새 세션(하네스 **Local**)에 오른쪽 한 줄 (키는 1-1 결과로)
2. 카드 확인: 브랜치 `feat/KAN-5` · Jira **In Progress** · 수정
3. 인수조건 ↔ 테스트 표 · 커밋 1개 확인
4. push 는 실습 2의 `/pr-create` 가

프롬프트 / 명령

```
/jira-implement KAN-5
```

✅ 브랜치 `feat/KAN-5` ✅ Jira In Progress · 구현 시작 코멘트 ✅ 커밋 `KAN-5 feat: …`

02

CI/CD — PR 을 열기 전에 워크플로부터

CI · CD 워크플로를 브랜치에 먼저 넣어 두면, PR 에서 CI 가 돌고 머지가 곧 배포가 됩니다.

최소 워크플로

.github/workflows/test.yml

```yaml
name: Tests

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest -q
```

1 **트리거**

push · pull_request 모두

2 **분량**

Copilot이 만든 16줄

3 **C#**

`actions/setup-dotnet@v4` (`10.0.x`) + `dotnet test -v q --nologo`

CD — 머지 뒤 배포, 두 가지 방식

	① 컨테이너 이미지	② 릴리스 파일 + 현장 수동 배포
맞는 곳	Docker 서버 + 레지스트리 접속 가능	폐쇄망 · 현장 장비
트리거	`main` 에 push (= PR 머지)	`v*` 태그 push (예: `v0.1.0`)
산출물	`ghcr.io` 이미지, 태그 **커밋 SHA** + `latest`	**Release** 에 `.tar.gz` + `SHA256SUMS`
권한	`packages: write`	`contents: write`
설치	`docker pull` → `docker run`	`scripts/install.sh`
정상 확인	`HEALTHCHECK` 가 `/health` 호출	`readlink current` + import + `/docs`
되돌리기	이전 **SHA 태그** 이미지로 `run`	`current` 링크를 이전 폴더로

① 컨테이너 배포 job

```
.github/workflows/cd-container.yml (배포 job)

  publish:
    needs: test                 # 테스트 job 통과 후에만
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write           # 이 job만 패키지 쓰기
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}   # 별도 시크릿 없음
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: |
            ghcr.io/${{ github.repository }}:${{ github.sha }}
```

② 릴리스 파일 — release.yml 흐름

1

v* 태그 push

예: `v0.1.0`, 권한 `contents: write`

2

pytest -q

테스트 통과 후에만 다음 단계

3

패키징

`tar -czf` 아카이브 + `sha256sum` → `SHA256SUMS`

4

GitHub Release

산출물 첨부

설치 스크립트: 체크섬 먼저 · 버전별 폴더 · 마지막에 `current` 전환 · 롤백 = 링크만

워크플로가 먼저 들어가 있으면

1

PR 열림 (push)

Tests 자동 실행 · Jira PR 열림

2

머지 (= main push)

Tests → **Publish container**

3

배포 성공

Jira deploy → Done

4

v* 태그 push

Release tarball → Releases

사람이 하는 일은 PR 승인 · 리뷰 판단 · 머지 · 태그 — 나머지는 워크플로가 이어 받음

실습

CI/CD 워크플로 준비 — PR 전에

test.yml · 컨테이너 배포 · 릴리스 파일을 Copilot 으로 만들고,
사람이 검토한 뒤 커밋 (push 는 PR 때)

준비

- 실습 1의 `feat/KAN-5` 브랜치 (커밋 1개 위)
- Docker (컨테이너 검증용, 없으면 2-2 검증 건너뜀)

실습 2-1

test.yml — CI 워크플로

1. 오른쪽 프롬프트 → 앞 장의 16줄과 비교
2. **Keep** — 아직 커밋하지 않음

프롬프트 / 명령

```
push 와 pull_request 마다 테스트를
실행하는 GitHub Actions 워크플로
.github/workflows/test.yml 을 만들어라.
Python 3.12,
pip install -r requirements.txt,
pytest -q. 캐시·매트릭스 없이 최소로.
파일만 만들고 커밋하지 마라.
```

✅ **16줄 test.yml**

실습 2-2

배포 ① 컨테이너

1. 오른쪽 프롬프트 — 파일만 만들고 커밋 금지
2. 다음 장 (3) 까지 이어 붙여 한 번에 전송

프롬프트 (1/2 — 다음 장과 한 번에 보냄)

```
배포 방식 ① 컨테이너 배포를 만들어라.
(1) 저장소 루트에 Dockerfile:
python:3.12-slim, requirements 설치 후
uvicorn equip_monitor.app:app 을
0.0.0.0:8000 으로 실행, 비루트 사용자,
HEALTHCHECK 로 /health 확인.
(2) .dockerignore.
```

실습 2-2

배포 ① 프롬프트 (이어서)

1. 앞 장 (1)(2) 뒤에 이어 붙여 한 번에 전송

2. `needs: test` · `packages: write` · `GITHUB_TOKEN` 확인

3. `docker build` · `docker run` 터미널 카드는 명령을 읽고 **Allow**

프롬프트 (2/2)

```
(3) .github/workflows/cd-container.yml:
main 에 push 되고 test 가 통과한 뒤에만
이미지를 빌드해 ghcr.io/<owner>/<repo> 에
커밋 SHA 태그와 latest 태그로 푸시.
워크플로 name 은 Publish container.
권한은 packages: write 만 추가하고
시크릿은 GITHUB_TOKEN 만 써라.
파일만 만들고 커밋하지 마라.
```

✅ 3 files ✅ `{"status":"ok"}` ✅ 사용자 app

실습 2-3

배포 ② 릴리스 파일 + 현장 설치 스크립트

1. 오른쪽 프롬프트 — 파일만 만들고 커밋 금지
2. 테스트 실행 파일을 못 찾으면 명령 직접 지정
3. 자체 검토로 체크섬 허점을 찾는지 확인

프롬프트 (1/2 — 다음 장과 한 번에 보냄)

```
배포 방식 ② 릴리스 파일 + 현장 수동 배포를
만들어라.
(1) .github/workflows/release.yml:
v* 태그가 push 되면 테스트 후
src·requirements.txt·pyproject.toml·
scripts/install.sh·INSTALL.md 를
equip-monitor-<태그>.tar.gz 로 묶고
SHA256SUMS 파일을 만들어
GitHub Release 에 첨부.
권한은 contents: write 만.
```

실습 2-3

배포 ② 프롬프트 (이어서)

1. 앞 장 (1) 뒤에 이어 붙여 한 번에 전송
2. 어느 폴더에서든 동작 · `PREFIX` 조건 확인
3. 임시 폴더 설치 → 두 번째 버전 → 롤백 결과 확인

프롬프트 (2/2)

```
(2) scripts/install.sh: tar.gz 체크섬을
SHA256SUMS 로 검증 → /opt/equip-monitor/<버전>
에 풀고 venv·의존성 설치 → current 링크 전환
(롤백 가능). 경로는 PREFIX 로 바꿀 수 있게,
어느 폴더에서 실행해도 설치 폴더 기준으로
(pip 은 설치 폴더에서). (3) INSTALL.md(한국어).
파일만 만들고 커밋하지 마라. 끝나면 PREFIX 를
저장소 밖 임시 폴더로 바꿔 설치 → 두 번째
버전 설치 → 롤백까지 직접 확인해라.
```

✅ **3개 파일** ✅ **bash -n 통과** ✅ **설치 · 롤백 통과**

실습 2-4

커밋 3개 — push 는 PR 때

1. 워크플로 · 배포 파일을 커밋 3개로 — 각각 한 줄
2. `git log --oneline` 으로 커밋 4개 확인 (실습 1 + 3)
3. **push 하지 않음** — 실습 3의 `/pr-create` 가

명령 (터미널, 한 줄씩)

```
git add .github/workflows/test.yml
git commit -m "ci: run tests on push and pull request"

git add Dockerfile .dockerignore .github/workflows/cd-container.yml
git commit -m "ci: build and publish container image on main"

git add .github/workflows/release.yml scripts/install.sh INSTALL.md
git commit -m "ci: package release tarball with checksums and add
on-site installer"
```

✅ 커밋 3개 추가 ✅ 아직 push 안 함

03

PR 만들기

PR 설명은 리뷰의 입력입니다. 초안은 Copilot이, 확인은 사람이 합니다.

좋은 PR 설명 — PR 템플릿

.github/pull_request_template.md

```
## 이슈
Closes #

## 변경 요약
-

## 테스트 방법
pytest -q
python scripts/check_rules.py data/samples/overheat.log

## Copilot 사용 기록 (실습용)
- 사용한 instructions/Skill/Agent:
- 요청 횟수(대략):
- AI 가 틀려서 사람이 고친 곳:
```

1 **Closes #1**

머지 시 이슈 자동 닫힘 — 이슈마다 한 줄씩 (Fixes도 동일)

2 **Jira 키 · Confluence 링크**

사람과 Copilot code review 모두의 단서

3 **Conventional Commits**

제목·커밋은 `feat:` · `fix:` 등 한 줄

설명 초안을 만드는 두 가지 방법

	VS Code — `/pr-create` Skill	GitHub 웹 — Copilot Summary
방법	초안 → 사람 승인 → push · PR 생성까지	설명란 툴바 Copilot 아이콘 → **Generate → Summary**
장점	템플릿 형식 · Jira 키 · 말투 지시 가능	버튼 하나, 파일별 변경 요약이 정확
단점	크레딧 사용, 단계 많음	**설명란 기존 내용 무시**, 영어로 출력
플랜	모든 플랜	Copilot Free 제외

템플릿 · Jira 키를 지키려면 `/pr-create`, 템플릿이 없으면 Summary가 빠름

실습

PR 만들기

`/pr-create` 로 템플릿에 맞는 초안을 보고,
승인하면 push · PR 생성까지

준비

- 실습 2까지 커밋된 `feat/KAN-5` (구현 + 워크플로)
- GitHub MCP · Atlassian MCP (하네스 **Local**)

실습 3-1

/pr-create — 초안 · 승인 · push · PR

1. 새 세션(하네스 **Local**)에 `/pr-create`
2. 초안 확인 — **base** `main` · 제목 `<Jira 키> feat: …` · `Closes #1` · Jira 칸
3. "승인" → push 카드 · PR 생성 카드 각각 **Allow**
4. PR 상단 `main ← feat/KAN-5` · 하단 **Tests** → ❌ (실습 4에서)
5. **Actions → Jira sync** ✅ → Jira `PR 열림`

프롬프트 / 명령

```
/pr-create

# 초안을 확인한 뒤
승인
```

✅ base `main` ✅ Tests ❌ (1 failed) ✅ Jira `PR 열림`

04

CI 가 잡은 실패 — 우회 말고 원인

PR 의 Tests 가 빨간불입니다. 내 PC 에서는 통과했는데 GitHub 러너에서는 실패합니다.

"내 PC에서는 되는데 CI에서는 실패"

내 PC(KST)에서는 통과, Actions(UTC)에서는 `1 failed`

원인 수정이 아님

- `env: TZ: Asia/Seoul` 추가
- `skipif(CI)` 로 건너뛰기
- 기대값을 UTC에 맞춤

원인 수정

- 원인: `.astimezone()` 이 **머신 시간대** 사용
- 수정: 시간대를 **인자로 명시**

실습

CI 실패 → 원인 수정 → ✅

PR 의 Tests 가 잡은 실패를 우회하지 않고
원인으로 고쳐 초록불로

준비

- PR 하단 **Tests** ❌
- 저장소 **Actions** 탭

실습 4-1

PR 의 Tests ❌ → 실패 로그

1. PR 하단 **Tests ❌ → Details**

2. 실패 작업 → *Run pytest -q* 로그

3. 마지막 줄(FAILED ...) 복사

✅ **1 failed (passed 수는 브랜치마다 다름)**

실습 4-2

원인만 묻기 (아직 수정 금지)

1. 로그 마지막 줄을 붙여 오른쪽 프롬프트
2. 원인이 실행 머신 시간대인지 확인
3. 우회책이면 "원인 수정이 아니다"로 재질문

프롬프트 / 명령

```
CI 에서 이 테스트만 실패한다.
로컬에서는 통과한다.
원인을 설명하고 수정안을 제시해라.
아직 코드는 바꾸지 마라.
FAILED tests/test_report.py::
test_day_range_includes_early_morning_kst
- AssertionError: ...
```

✅ **원인: `uptime.py:25 .astimezone()`** ✅ **코드 변경 없음**

실습 4-3

원인 수정 → Actions ✅

1. 수정 요청 — 테스트 · CI 변수는 그대로
2. UTC · 로컬 두 시간대 결과 확인
3. 커밋 · push → Actions ✅

프롬프트 / 명령

```
그 방향으로 고쳐라.
day_range 에 tz 인자를
받게 하고(기본 Asia/Seoul),
테스트를 바꾸거나
CI 환경 변수를 바꾸지 마라.
끝나면 python -m pytest -q 를
UTC 시간대(Windows PowerShell:
$env:TZ="UTC0", mac: TZ=UTC)와
로컬 시간대에서 둘 다 돌려라.
```

✅ **UTC · 로컬 모두 통과 (failed 0)** ✅ **Actions** ✅

결과 — Copilot이 찾은 실패 원인

Completed 8 steps in 32s

원인

`uptime.py:25` 는 timezone 없는 자정 시각에 `astimezone()` 을 호출합니다. 이 호출은 `day` 의 시간대로 변환하는 게 아니라 **실행 머신의 로컬 시간대**를 사용합니다. 따라서 UTC인 CI에서는 시작이 `2026-09-21 00:00 UTC` 가 되지만, 로컬 머신이 KST면 `2026-09-21 00:00 KST` 가 되어 테스트 결과가 달라집니다.

테스트는 9월 21일 KST 기준 하루를 기대합니다(`test_report.py:14`). 비교 대상인 `05:00 KST` 는 `20:00 UTC` (전날)이므로 CI의 시작 시각 `00:00 UTC` 보다 앞섭니다. 그래서 그 시각이 리포트 구간에서 제외되어 실패합니다. 즉, `day_range` 의 구현과 테스트의 KST 기대가 서로 다르고, 결과가 머신의 시간대에 의존합니다.

수정안

리포트 날짜의 기준을 KST로 명시하고, KST 자정과 다음 날 자정을 각각 만든 뒤 UTC로 변환하는 것이 적절합니다. 그러면 반환값도 프로젝트의 UTC datetime 규약을 따르며, 실행 환경과 무관하게 같은 날짜 구간을 계산합니다. 문서 주석도 "머신 로컬 시간대"에서 "KST" 기준으로 바꾸고, 검증은 UTC 환경에서 해당 테스트를 실행해 재현 여부를 확인하면 됩니다.

코드는 수정하지 않았습니다.

1 **원인**

`uptime.py:25` 의 `astimezone()` — 실행 머신 시간대 사용

2 **그래서 CI만 실패**

UTC 러너에선 KST 구간이 어긋남

3 **수정안**

KST 기준을 명시하고 UTC로 변환 — 코드는 아직 안 고침

05

Copilot code review

리뷰는 매번 다른 결과를 냅니다. 지적을 분류하고, instructions로 범위를 좁힙니다.

code review가 남기는 것

1. **코멘트 + 제안 수정**
   High / Medium / Low 심각도 표시

2. **Copilot review overview**
   결론(Changes / Approval recommended) · 수준 · 지적 목록

3. **"Comment" 리뷰가 기본**
   필수 승인 수에 미포함 (승인 반영은 프리뷰)

4. **수준 Lite · Balanced**
   Balanced 기본. 1회 $0.05~$5 어치 크레딧 + Actions 분

알아 둘 점

Copilot 코멘트에 단 **답글은 Copilot이 읽지 않음** — 사람끼리의 기록

리뷰는 **PR 브랜치**의 지침 · Skill 기준

실습

리뷰 요청 · 반영 · 범위 좁히기

Copilot 리뷰를 받는 동안 flow-tracker 로 흐름을 점검하고,
지적을 판단해 반영한 뒤 instructions로 범위를 좁힘

준비

- 실습 4에서 Tests ✅ 가 된 PR
- 새 세션에서 하네스 **Local** (Atlassian · github MCP)

실습 5

Copilot 리뷰 요청 + `flow-tracker` 로 점검

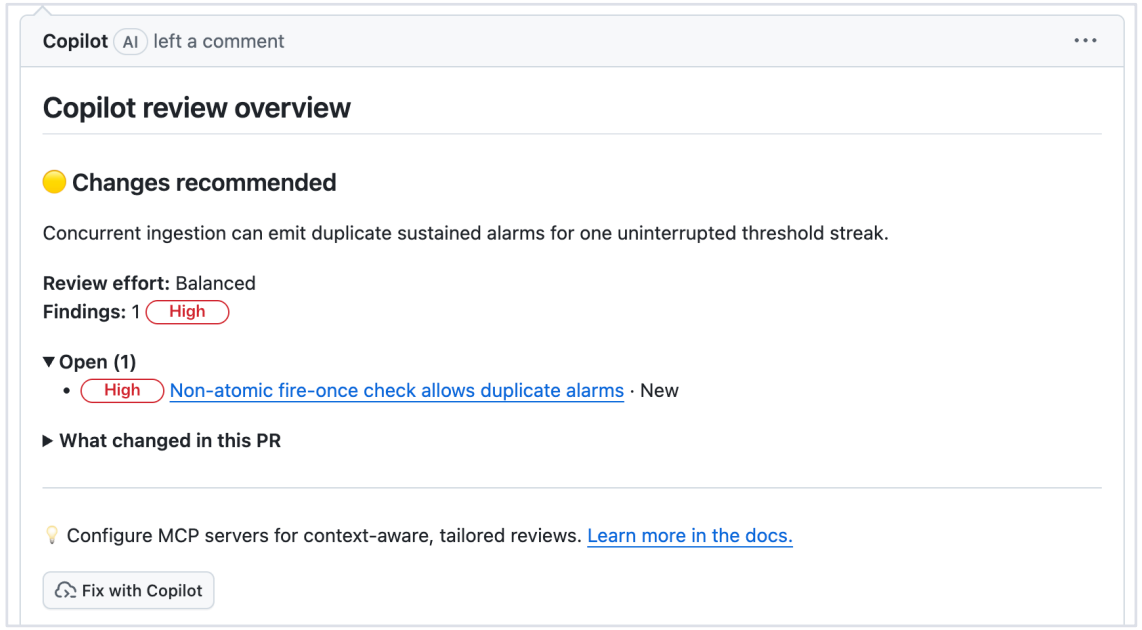
1. **Reviewers** → 수준 **Balanced** 확인 → **Request**
2. 노란 점 = 리뷰 중 (약 3분)
3. 기다리는 동안 에이전트 드롭다운 → **flow-tracker**
4. 오른쪽 한 줄 — 쓰기 카드가 뜨면 안 됨

프롬프트 / 명령

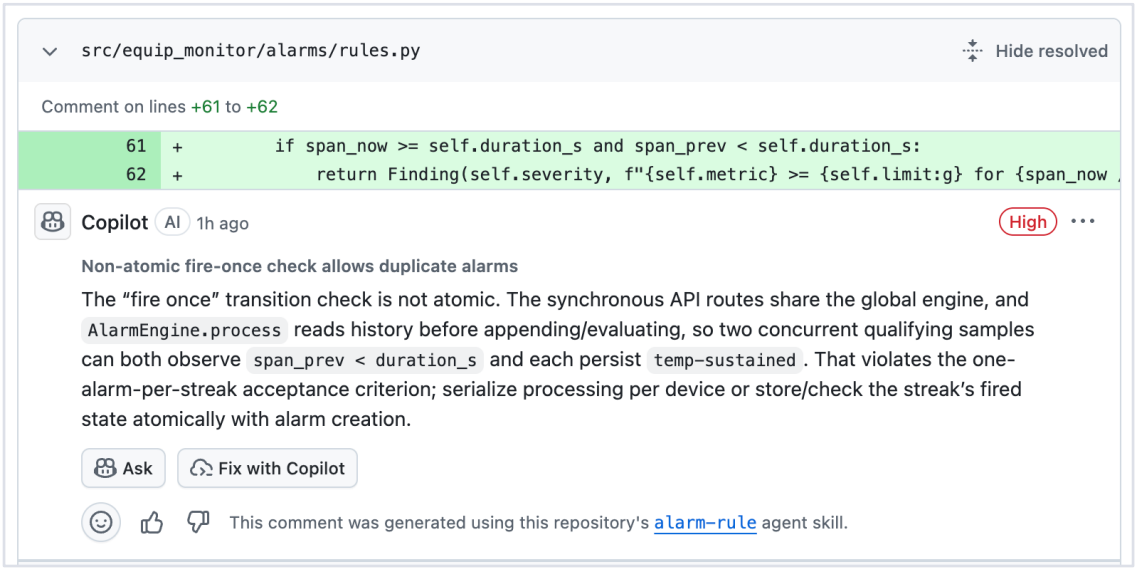
```
KAN-5
```

✅ 단계별 표 (Jira · PR · 리뷰 · CI …) ✅ 쓰기 도구 호출 없음 ✅ 리뷰 결과 도착

결과 — 리뷰 개요와 코멘트



Copilot review overview(리뷰 개요)



줄 단위 코멘트 — 심각도 · Ask · Fix with Copilot

1. **결론 · 수준 · 지적 수**
   Changes recommended, Review effort, Findings
2. **코멘트마다 심각도**
   High 표시, 제목 링크로 해당 줄 이동
3. **Skill 사용 표시**
   "alarm-rule agent skill" 문구

리뷰 코멘트는 3분류로

분류	처리
맞음	고치기 → 커밋 → 재리뷰
틀림	근거를 답글로 남기고 **Resolve conversation**
취향 · 범위 밖	이번 PR 제외, 필요하면 별도 이슈. 반복되면 **instructions**로 범위 좁히기

재리뷰는 같은 결과를 내지 않음 — 결론은 사람이 내고 Resolve로 기록

리뷰 범위를 좁히는 instructions 예

.github/copilot-instructions.md (끝에 추가)

```
## Code review
리뷰어(Copilot code review)는 다음만 본다:
① naive datetime·로컬 시간대 의존
② 경계 비교(`>=`/`>`)와 경계 테스트 유무
③ 라우터의 store 내부 접근.
포매팅·네이밍 취향·docstring 문체는 지적하지 않는다.
성능·동시성은 이 실습 범위
(단일 프로세스, 메모리 저장소) 밖이므로 지적하지 않는다.
```

1 **무엇을 볼지 · 무시할지**

Instructions에는 이 두 가지만 작성

2 **형식 지시는 미지원**

코멘트 형식 · PR overview 변경 불가
(Unsupported instruction types)

3 **PR 브랜치에서 바로 적용**

instructions를 고친 그 PR의 재리뷰부터 반영

실습 6-1

지적 판단 → 최소 수정

1. 지적이 맞는지 **먼저 판단** 후 맞을 때만 수정
2. 테스트 파일은 건드리지 않게 지시
3. `pytest` 를 못 찾으면 **Skip** → 명령 지정
4. `threading.Lock` 최소 수정 · 전체 통과

프롬프트 / 명령

```
github MCP 로 내 PR(#<번호>)의 Copilot 리뷰
코멘트(Non-atomic fire-once check)를 읽고,
코드에서 지적이 맞는지 먼저 판단해라.
맞다면 AlarmEngine.process 를 직렬화하는
최소 수정만 하라. 테스트 파일은 건드리지
말고, 끝나면 python -m pytest -q 전체를
돌려 결과를 보여줘.

# pytest 를 못 찾을 때
테스트는
.venv\Scripts\python -m pytest -q
(mac: .venv/bin/python -m pytest -q)
로 실행해라.
```

실습 6-2

재리뷰 → 3분류 → instructions로 범위 좁히기

1. 6-1 수정 커밋 · push → Copilot 옆 🔄 로 **재리뷰**
2. 새 지적 · Previously missed를 3분류로 판단
3. instructions 끝에 `## Code review` 절
4. 커밋 · push → 🔄 재리뷰
5. 남은 Open은 3분류대로 **Resolve conversation**

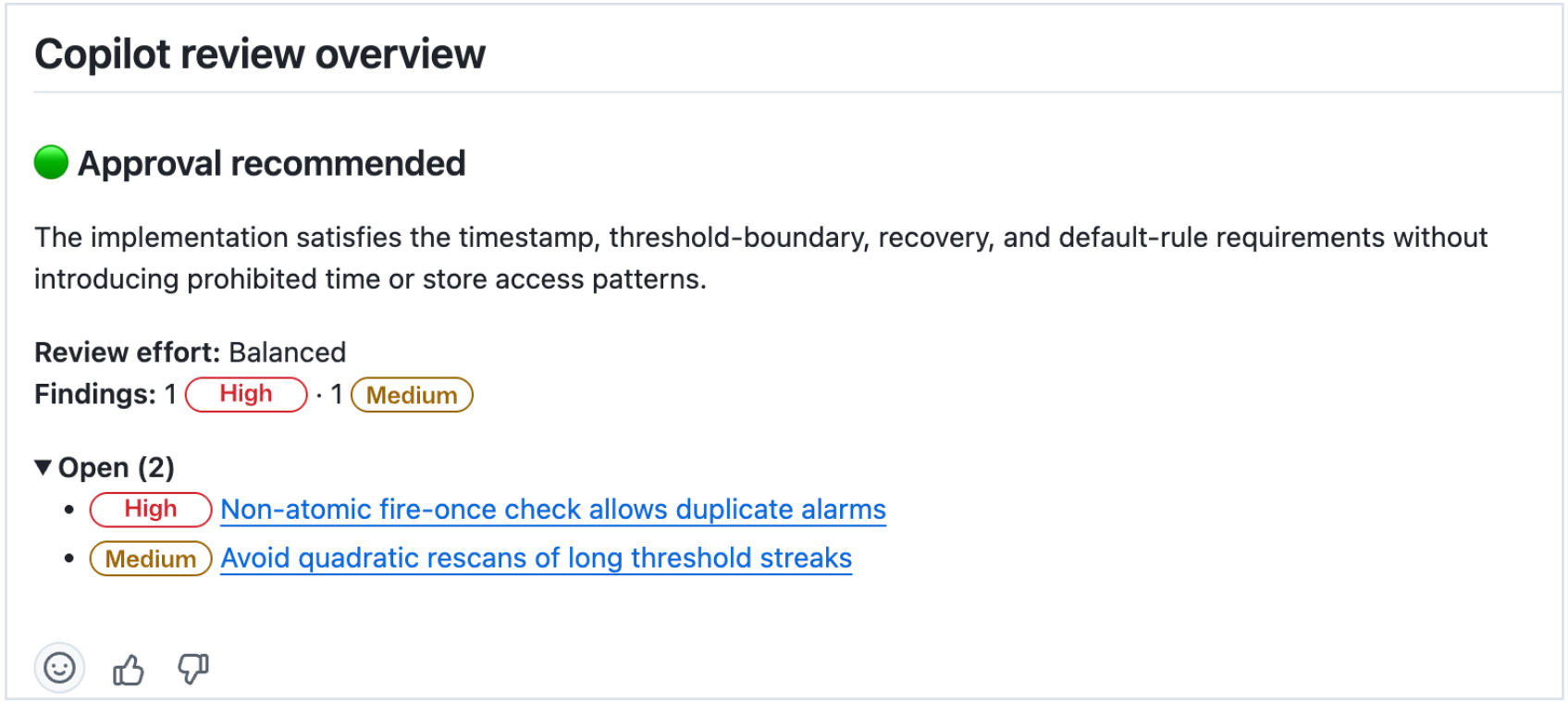
명령 (commit 은 각각 한 줄)

```
# 6-1 수정 반영
git commit -am "fix: serialize alarm engine
processing to prevent duplicate alarms"
git push

# instructions 추가 후
git commit -am "docs: scope Copilot code review
to time, boundary, and store rules"
git push
```

✅ **Approval recommended** ✅ **새 지적 없음** ✅ **Open 0개**

결과 — instructions 추가 뒤의 리뷰



Copilot review overview

🟢 Approval recommended

The implementation satisfies the timestamp, threshold-boundary, recovery, and default-rule requirements without introducing prohibited time or store access patterns.

Review effort: Balanced
Findings: 1 High · 1 Medium

▼ **Open (2)**

- High Non-atomic fire-once check allows duplicate alarms
- Medium Avoid quadratic rescans of long threshold streaks

1 **Review effort: Balanced**
리뷰 수준 · 결론 표시

2 **지적은 범위 안**
중복 알람(High) · 반복 스캔(Medium)

3 **취향 지적 없음**
포매팅 · 네이밍 · 문체 지적 사라짐

06

머지 → 배포 → Jira Done

머지는 사람이 합니다. 그다음은 PR 전에 넣어 둔 워크플로가 배포하고 Jira 를 Done 으로 바꿉니다.

실습

머지 → 자동 배포 → Jira Done

사람이 머지하면 워크플로가 배포하고 Jira 를 Done 으로 —
어디서 무엇을 확인하는지 하나씩, 태그를 push 하면 Release 까지

준비

- Tests ✅ · 리뷰 반영된 PR
- 저장소 **Actions** 탭 · Jira 작업 화면을 탭으로 열어 두기

머지 뒤에 자동으로 일어나는 일 — 어디서 보나

순서	무엇이 (트리거)	어디서 확인	걸린 시간*
① **머지**	`main` push → **Tests** · **Publish container** 시작	PR 하단 · **Actions** 탭	즉시
② **배포**	**Publish container**: test → publish (GHCR 이미지)	**Actions** → Publish container 실행	약 1분
③ **이미지**	`ghcr.io/<계정>/<저장소>` 태그 `latest` · 커밋 SHA	저장소 메인 오른쪽 **Packages**	②와 함께
④ **Jira**	**Jira deploy** (Publish container 성공에 반응) → Done	**Actions** → Jira deploy · Jira 작업	수십 초
⑤ **릴리스**	`v*` 태그 push → **Release tarball**	저장소 오른쪽 **Releases**	약 30초

* 리허설 기준. 사람이 하는 일은 ① 머지 버튼과 ⑤ 태그 push 두 번뿐

실습 7-1

머지 — 사람이

1. PR 상단이 `main ← feat/KAN-5` 인지 (**base `main`**)
2. **All checks have passed · No conflicts** 확인
3. **Merge pull request → Confirm merge → Delete branch**
4. **Actions**: `main` 의 `Tests` → **Publish container** 실행
5. Jira: 머지됨 – 배포 대기 (아직 Done 아님)

✅ **base `main` 에 머지** ✅ **이슈 #1 자동 닫힘** ✅ **Publish container 시작**

실습 7-2

Actions 에서 배포 지켜보기 — Publish container

1. 저장소 **Actions** 탭 → 왼쪽 목록 **Publish container**
2. 맨 위 실행(브랜치 `main` · 머지 커밋 메시지) 클릭
3. job **test** ✅ → **publish** ✅ 순서 확인
4. **publish** 펼쳐 *Build and push* 로그에 이미지 태그 확인

✅ **Publish container** ✅ **(main)** ✅ **test → publish 순서**

실습 7-3

배포 결과 — Packages 의 이미지

1. 저장소 메인(Code 탭) 오른쪽 사이드바 **Packages**
2. 패키지 열기 → 태그 `latest` · 커밋 SHA
3. (선택) 내 PC 에서 받아 `/health`

명령 (선택, 터미널 한 줄씩 — OS 에 맞는 docker run 하나만)

```
# 비공개 패키지일 때만 로그인
gh auth token | docker login ghcr.io -u <계정> --password-stdin

# mac (Apple Silicon) — --platform 필요
docker run -d --name em --platform linux/amd64 \
  -p 8000:8000 ghcr.io/<계정>/<저장소>:latest
# Windows (x64, PowerShell) — --platform 없이
docker run -d --name em -p 8000:8000 `
  ghcr.io/<계정>/<저장소>:latest

docker logs em
curl http://localhost:8000/health    # Windows: curl.exe
docker rm -f em
```

✅ **Packages 에 이미지** ✅ **태그 latest + SHA** ✅ **(선택) {"status":"ok"}**

실습 7-4

Jira 가 Done 으로 — Jira deploy

1. **Actions** → 왼쪽 **Jira deploy** → 맨 위 실행 ✅

2. 로그 마지막 줄 `KAN-5 → Done`

3. Jira 작업 새로고침: 상태 **Done**

4. 코멘트 `배포 완료: ghcr.io/…:<SHA>` · 라벨 `deploy-pending` 사라짐

✅ **Jira deploy** ✅ ✅ **Jira Done (사람 손 없이)** ✅ **`deploy-pending` 해제**

실습 7-5

flow-tracker 로 전체 흐름 점검

1. 새 세션 → 에이전트 드롭다운 **flow-tracker**
2. 오른쪽 한 줄 (내 Jira 키)
3. Confluence → Jira → PR → 리뷰 → CI → 머지 → 배포 → Done 모두 ✅
4. ⏳ · ❌ 칸 → 표 아래 다음 할 일대로

flow-tracker 에 보낼 한 줄

```
KAN-5
```

✅ 표 전부 ✅ ✅ 쓰기 도구 호출 없음

실습 7-6

태그 → Release — 현장 배포용 파일

1. 로컬 `main` 최신화 (머지된 커밋 받기)
2. 태그 `v0.1.0` 만들고 push
3. **Actions → Release tarball** ✅
4. 저장소 오른쪽 **Releases** → `v0.1.0` 첨부 2개 확인

명령 (터미널, 한 줄씩)

```
git switch main
git pull
git tag v0.1.0
git push origin v0.1.0
```

✅ **Release v0.1.0** ✅ **tar.gz · SHA256SUMS**

실습 7-7

(선택) Release 파일로 설치해 보기 — mac / Linux

1. Releases 의 두 파일을 빈 폴더에 내려받기
2. `PREFIX` 를 임시 폴더로 바꿔 설치 → `readlink` 가 `v0.1.0`
3. `current` 의 venv 로 실행 → 다른 터미널에서 `/health`
4. Ctrl+C 로 끄고 임시 폴더 정리

명령 (mac / Linux — bash · zsh · fish 공통)

```
mkdir -p /tmp/em-site
cd /tmp/em-site
gh release download v0.1.0 -R <계정>/<저장소>
tar -xOf equip-monitor-v0.1.0.tar.gz scripts/install.sh \
  | env PREFIX=/tmp/em-site/opt \
    bash -s -- equip-monitor-v0.1.0.tar.gz
readlink /tmp/em-site/opt/current

# 실행 (8000 은 docker 와 겹칠 수 있어 8001)
/tmp/em-site/opt/current/venv/bin/uvicorn \
  equip_monitor.app:app --port 8001
# 다른 터미널 → 확인 뒤 실행 터미널에서 Ctrl+C
curl http://127.0.0.1:8001/health
rm -r /tmp/em-site
```

✅ **current → v0.1.0** ✅ **{"status":"ok"}**

실습 7-7

(선택) Release 파일로 설치해 보기 — Windows

1. PowerShell 에서 빈 폴더에 두 파일 내려받기
2. Linux 컨테이너를 현장 장비로 띄우기
3. 컨테이너 안에서 설치 · `readlink` · 실행
4. PowerShell 에서 `curl.exe` 로 `/health` → `exit` 로 정리

명령 (Windows — PowerShell + Docker)

```
# PowerShell
mkdir C:\temp\em-site; cd C:\temp\em-site
gh release download v0.1.0 -R <계정>/<저장소>
docker run --rm -it -p 8001:8001 -v "${PWD}:/site" `
  -w /site python:3.12-slim bash

# 컨테이너 안 (bash)
tar -xOf equip-monitor-v0.1.0.tar.gz scripts/install.sh \
  | env PREFIX=/opt/em bash -s -- equip-monitor-v0.1.0.tar.gz
readlink /opt/em/current
/opt/em/current/venv/bin/uvicorn equip_monitor.app:app \
  --host 0.0.0.0 --port 8001

# 새 PowerShell 창 → 확인 뒤 Ctrl+C, exit
curl.exe http://localhost:8001/health
```

✅ **current → v0.1.0** ✅ **{"status":"ok"}**

설치된 모습 — 버전별 폴더와 current 링크

```
/tmp/em-site  (PREFIX=/tmp/em-site/opt)

/tmp/em-site/
├── equip-monitor-v0.1.0.tar.gz   # 받은 릴리스 파일
├── SHA256SUMS                    # 아카이브 옆에 둠
└── opt/                          # PREFIX
    ├── v0.1.0/                   # 버전별 폴더
    │   ├── src/ · requirements.txt · INSTALL.md …
    │   └── venv/                 # 이 버전 전용 파이썬 환경
    └── current -> v0.1.0         # 지금 쓰는 버전

# 실행 — 설치는 준비까지, 실행은 current 의 venv 로
/tmp/em-site/opt/current/venv/bin/uvicorn \
  equip_monitor.app:app --port 8001
# 다른 터미널
curl http://127.0.0.1:8001/health
# {"status":"ok"} 확인 뒤 Ctrl+C
```

1 **버전별 폴더**

새 버전은 옆에 새 폴더, 이전 버전은 그대로

2 **current 링크**

실행은 늘 `current/…` · 전환은 링크로

3 **롤백**

`install.sh --rollback v0.1.0` → 링크만 되돌림

4 **현장에서는**

`PREFIX=/opt/equip-monitor` + sudo, 서비스 등록은 별도

핵심 요약

1. 요구사항은 Confluence에 먼저 — `/req-to-jira` Skill 이 요구사항 글만 Jira 로, 양방향 링크까지
2. 구현은 `/jira-implement` — `feat/<Jira 키>` 브랜치 · Jira 진행 중 · 테스트 먼저 · 커밋까지
3. CI/CD 워크플로는 PR 전에 — PR 에서 CI 가 돌고, 머지가 곧 배포, 배포 성공이 곧 Done
4. PR 설명은 리뷰의 입력 — 템플릿 + `Closes #N` + Jira 키 · Confluence 링크
5. 리뷰는 매번 다름 — 맞음 / 틀림 / 취향으로 나누고 `## Code review` instructions로 범위 좁히기
6. CI 실패는 원인 수정 — 환경 변수 · skip으로 덮기 금지, 설치 스크립트는 다른 폴더 설치까지 요구사항에
7. 머지는 사람이 (hook 이 에이전트 차단) — Jira 상태는 워크플로가 PR 열림 → 배포 대기 → Done